# anwuRAW v2.1.22 · Android 端「功能模块」Bug 排查报告

- 排查对象：`github.com/Tri250/anwuRAW` tag `v2.1.22`（2026-11-08 发布）
- Android 产物：`04_RapidRAW_v2.1.22_android_arm64-v8a.apk` / `.aab`
- 排查方式：静态源码审计，方法论遵循 debug-pro 七步协议
- 排查日期：2026-11-12
- **本报告聚焦「功能模块」层面**（上一份报告覆盖签名/构建/JNI/安全等基础设施层，两份互补不重复）

## 结论

**共定位 24 项功能性缺陷**，分布于 9 个功能模块：

| 模块 | 代码位置 | 缺陷数 |
|---|---|---|
| 跨模块（全局机制） | `crystalShared.tsx`、`AndroidManifest.xml`、`useEditorActions.ts` | 5 |
| 构图（compose） | `CrystalCategoryPanel.tsx:1678-1979` | 3 |
| 预览 / 手势 / 对比 | `CrystalPreview.tsx`（769 行） | 5 |
| 导出（水印导出域） | `WatermarkExportPage.tsx`（557 行） | 5 |
| 人像（portrait） | `CrystalCategoryPanel.tsx:1027-1678` | 3 |
| 编辑壳 / 底栏导航 | `CrystalMobileApp.tsx`（460 行） | 2 |
| 滑杆基础组件 | `crystalShared.tsx: CrystalRangeSlider` | 2 |
| EXIF 信息栏 | `crystalShared.tsx: summarizeExif` | 1 |
| 偏好持久化 | `crystalShared.tsx: useWmPrefs` | 1 |

按严重度：**P0 阻断 3 项 / P1 严重 9 项 / P2 中等 8 项 / P3 轻微 4 项**。

另有 **3 项存疑待验证**（需真机或进一步读码确认），单列在末尾，不计入 24。

## 排查边界（务必先看）

debug-pro 第 1 步 **Reproduce 未完成**：本次为纯静态审计，**未在真机/模拟器运行**，第 5 步 Verify 未执行。所有结论基于代码路径推导 + 交叉对照证据。

移动端功能层总计 8271 行（7 个文件），其中 `CrystalCategoryPanel.tsx` 2993 行单文件承载人像/调色/构图/特效四个模块。**受篇幅与上下文限制，调色（TonePanel，227-1027 行，约 800 行）与特效（FxPanel，2009-2940 行，约 930 行）两个模块只做了定点抽查（深度图 busy 状态机、LUT 加载），未做逐行审计**。 这两个模块是本次排查覆盖最薄的部分，不能声称「已 100% 排查」。

已抽查确认**没有问题**的部分（避免误报）：

- 特效/人像的深度图生成 busy 状态机（`CrystalCategoryPanel.tsx:1440-1478`）——守卫置于 busy 置位之前、`finally` 里 dismiss toast + 复位 busy、跨图丢弃结果，写得是对的

- `awaitImageReady`（`crystalShared.tsx:236`）本身有「已就绪则永等到 10s 超时」的设计陷阱，但**全部 8 处调用点都有 `!isReady &&` 前置守卫**，实际不触发
- 导出页 PNG 模式的画质档——`crystal.css:1831` 有 `pointer-events: none`，是真禁用，不是假禁用
- Android 导出不存在覆盖源文件风险——后端固定写 `Pictures/RapidRaw`（MediaStore），与源目录不同
- 前端传的 `exif_frame` 字段在 Rust 端**确实存在并被应用**（`export_processing.rs:153` / `:1378`），前端注释「旧后端会忽略」已过时但无害

---

## 一、跨模块缺陷（5 项）

### G-1 · P1 · 全仓未声明 VIBRATE 权限，Android 上所有触觉反馈静默失效

- 证据链：
  - `crystalShared.tsx` → `export const haptic = (pattern) => { try { navigator.vibrate?.(pattern); } catch {} }`
  - `haptic()` 在移动端功能层被调用 30+ 处：分类切换（`CrystalMobileApp.tsx:88`）、原图长按（`crystalShared.tsx: useShowOriginalHold`）、裁剪手柄按下、双击缩放、导出确认、模板选择、开关切换（`CxSwitch`）
  - `grep -rn "VIBRATE" src-tauri/` → **零命中**
  - `gen/android/app/src/main/AndroidManifest.xml` 只声明了 `android.permission.INTERNET`
- 根因：Android WebView 的 `navigator.vibrate` 底层走 `Vibrator` 服务，**必须持有 `android.permission.VIBRATE`**。缺失时调用不抛异常、只返回 false，被 `haptic` 的 `try/catch` 完全吞掉。
- 影响：Android 端**整套触觉反馈体系完全失效**，且因为静默失败，开发和测试都不会发现。这是跨全部七大功能模块的一致性缺陷——设计稿里「按下有震感」的反馈在真机上一个都不存在。
- 修复：`AndroidManifest.xml` 增加 `<uses-permission android:name="android.permission.VIBRATE" />`（普通权限，无需运行时申请）。

### G-2 · P0 · base64 大字段进撤销栈与 sidecar，移动端内存与 IO 双重放大

- 证据链：
  - `utils/adjustments.ts:330` → `lensBlurDepthMap: string | null`（`Adjustments` 的普通字段）
  - `CrystalCategoryPanel.tsx:1466` → `applyPatch({ lensBlurDepthMap: b64, ... })`，b64 是 `generate_full_image_depth_map` 返回的**全图深度图 base64**
  - `crystalShared.tsx` → `maskReady` 判定 `container?.subMasks[0]?.parameters?.maskDataBase64`，即 **AI 蒙版位图也以 base64 存在 `adjustments.masks` 内**
  - `useEditorActions.ts:30-32` → `debouncedSetHistory = debounce((newAdj) => pushHistory(newAdj), 500)`，历史栈存**完整 adjustments 快照**，`CrystalMobileApp.tsx` 注释明确是「50 步撤销栈」
  - `utils/adjustments.ts:769` → `lensBlurDepthMap: loadedAdjustments.lensBlurDepthMap ?? ...`，即该字段**会从 sidecar 读回**，也就是会被写进 sidecar
  - `grep` 确认 `useEditorStore.ts` / `useEditorActions.ts` / `adjustments.ts` 中**没有任何剥离大字段的逻辑**

- 影响（三重放大）：
  1. **内存**：一张全图深度图 base64 约 1–8 MB。50 步历史 × 每步一份完整快照 = 峰值可达数百 MB。Android 应用堆上限通常 256–512 MB → 开启镜头模糊或 AI 蒙版后连续调参，**极可能 OOM 崩溃**
  2. **IPC**：每次拖动任何滑杆都会 `setAdjustments` → 整个 adjustments（含深度图 base64）序列化过 Tauri IPC 到 Rust。**改一个曝光值要搬一次几 MB 的字符串**
  3. **磁盘**：sidecar JSON 被深度图 base64 撑到数 MB，`debouncedSave`（300ms）每次自动保存都重写整个文件
- 这是移动端最可能导致「用着用着就崩/越用越卡」的功能缺陷，且桌面端因为内存宽裕不易暴露，属于典型的「同一份代码在移动端才致命」。
- 修复：把 `lensBlurDepthMap` / `maskDataBase64` 从 `Adjustments` 中移出，改为独立的资源缓存（按图片路径 + 内容哈希索引，存文件而非内存），`Adjustments` 里只留引用 ID；`pushHistory` 与 sidecar 序列化前显式剥离这两个字段。

### G-3 · P1 · `activeTools` 只增不减，工具一旦激活永不关闭

- 证据：`crystalShared.tsx` → `useCrystalAdjust.applyPatch`

```
const tools = opts?.enableTools === false ? [] : getToolOwnersForKeys(keys);
setAdjustments({
  ...prev, ...patch,
  ...(tools.length
    ? { activeTools: { ...getActiveTools(prev), ...Object.fromEntries(tools.map((id)
=> [id, true])) } }
    : {}),
});
```

`Object.fromEntries(tools.map((id) => [id, true]))` —— **只写入 `true`，全流程没有任何地方写 `false`**。`applyPresetObject` 同样。 - 影响：用户把某个参数调回默认值后，该参数所属工具仍留在 `activeTools` 中处于激活态。累积效应是一次会话里用过的所有工具全部常驻激活。**若渲染管线按 `activeTools` 决定跑哪些算子，则表现为「调回默认了效果还在」+「越编辑越卡」**。 - 待确认项：我未读 `getActiveTools` / `getToolOwnersForKeys` 的实现，也未确认 GPU 管线是否以 `activeTools` 为算子开关。若管线改为按「参数是否等于默认值」判定，则本条降级为「状态字段脏但不影响渲染」。**这一条需要验证后再定级**，但代码层面的单向写入是确定的。

### G-4 · P2 · 返回键退出层因频繁重注册导致 LIFO 顺序错乱

- 证据：`CrystalMobileApp.tsx:334-370` 的 `useEffect` 里 `pushExitLayer(handler)`，deps 为 `[cancelImmersive, flow, isCompareOpen, isExporting, isResetConfirmOpen, onBackToLibrary, osPlatform, setCompareOpen, t]` —— **9 项，其中 `t`（i18n 函数）在语言切换/初始化时会变，`isExporting`、`flow` 高频变化**。 同时 `CrystalPresetsPanel.tsx:792`、`WatermarkExportPage.tsx:155`、`SmartAlbumModal.tsx:71`、`SurveyView.tsx:64`、`CrystalPresetsPanel/WatermarkExportPage` 各自也 push。
- 根因：`androidBackRegistry` 用数组 `push/pop` 维护 LIFO，`removeExitLayer` 用 `lastIndexOf` 定位。任一组件因 deps 变化重注册（remove + push），它就会**跳到栈顶**，原本在它之上的层被压到下面。
- 影响：多个浮层同时打开时（例如导出页正在导出 + 编辑壳的确认弹窗），返回键可能先关掉错误的层。用户感知为「返回键行为不稳定/有时要点两次」。
- 修复：给每个退出层一个稳定的 `id`，registry 用 `Map<id, handler>` + 独立的层级序号，注册与更新分离，避免重注册改变顺序。

### G-5 · P2 · 移动端水印偏好存 localStorage，与桌面设置体系割裂

- 证据：`crystalShared.tsx` → `PREFS_KEY = 'crystal.watermark.prefs.v1'`，`loadWmPrefs` / `useWmPrefs` 全部走 `localStorage`，注释明说「与桌面 appSettings 解耦，避免 Android 端大对象写入」。
- 影响：签名文案、字体族、字号、文件名后缀、收藏模板、默认模板这 6 项用户配置
  - 不进 `settings.json`，**换机/重装即丢**
  - 不参与桌面端已有的云同步
  - Android 系统「清除缓存」或 WebView 存储被回收即丢
  - 而 `Cargo.toml` 里 `tauri-plugin-store = "2.5.0"` **已经是依赖**，却没用在这上面
- 修复：改用 `tauri-plugin-store`（落 app data 目录，跟随备份策略），或并入 `appSettings`。

---

## 二、构图模块（3 项）

### C-1 · P1 · 9:16 竖幅 chip 永远无法高亮，选竖构图时高亮跳到 16:9

- 证据：`CrystalCategoryPanel.tsx:1679-1686` 定义顺序为 `[free(null), 1, 4/3, 3/2, 16/9, 9/16]`；`:1711-1719` 的高亮匹配：

```
const hit = ASPECT_CHIPS.find(
  (c) => c.value && (Math.abs(c.value - ratio) < 0.01 || Math.abs(1 / c.value -
ratio) < 0.01),
);
return hit ? hit.value : ratio;
```

- 逐项验算 `ratio = 9/16 = 0.5625`：
  - `16/9 = 1.7778` → `|1.7778 - 0.5625| = 1.215` 不命中；`|1/(16/9) - 0.5625| = |0.5625 - 0.5625| = 0` → **命中**
  - `find()` 返回第一个命中项，而 `16/9` 在数组中排在 `9/16` **之前**
- 结论：`activeAspect` 返回 `16/9`，于是 `16:9` 的 chip 拿到 `is-active`，而用户实际选中的 `9:16` chip **永远点不亮**。
- 同一 bug 被**复制了两份**：`CrystalMobileApp.tsx:170-177` 的 `IMMERSIVE_ASPECT_CHIPS.find(...)` 是完全相同的逻辑（注释还写明「与构图面板 activeAspect 同源，枚举镜像」），所以沉浸裁剪浮层里同样错。
- 根因：正/逆双向匹配是为了处理「旋转 90° 后比例取逆」（4:3 → 3:4）的场景，但没有排除**逆值本身也是独立 chip** 的情况。
- 修复：先做精确匹配（只比 `c.value === ratio`），无命中再做逆匹配；或把 chip 数组按「互为倒数」分组，匹配时优先同组内的正向值。两处逻辑应抽成同一个共享函数，消除镜像复制。

### C-2 · P2 · 对比视图下完全无法缩放

- 证据：`CrystalPreview.tsx:461` → `onPointerDown` 首行 `if (compareRef.current || !urlRef.current) return;`；`onWheel` 同样 `if (compare || !urlRef.current) return;`
- 影响：进入前后对比分割视图后，双指缩放、双击缩放、滚轮缩放**全部被禁用**，用户无法放大核对细节差异——而这恰恰是「对比」这个功能最主要的使用动机（看降噪/锐化/磨皮的局部差别）。只能拖动分割线看全图。
- 属于功能缺失。若为有意设计，应在 UI 上给出说明；否则应允许对比态下缩放（分割线的 `clipPath` 是按容器百分比算的，缩放时需同步换算，这是它被禁用的可能原因）。

### C-3 · P2 · 点对比按钮到画面出现之间无任何反馈，且无 loading 态

- 证据：`CrystalPreview.tsx:729` → 对比层的渲染条件是 `{compare && url && originalPreview && (...)}`。`originalPreview` 由主管线在 `compareView !== 'off'` 时**异步渲染**产出（见 `CrystalMobileApp.tsx:296-306` 的 `setCompareOpen` 注释）。
- 影响：用户点「对比」→ `isCompareOpen` 立即变 true、按钮态立即变化，但**画面上什么都不发生**，要等主管线把原图渲染完（低端机 + 大 RAW 可能 1–2 秒）对比层才出现。中间**没有任何 loading 指示**。用户会以为按钮坏了并反复点击。
- 对照：同文件在其它异步渲染场景是有 `hasRenderedFirstFrame` / `Loader2` 处理的，唯独对比层缺。
- 修复：`compare && !originalPreview` 时渲染一个占位 loading 层。

---

## 三、预览 / 手势模块（5 项）

### P-1 · P1 · `imgRect` 没有绑定图片 `load` 事件，三分网格与裁剪框可能永不显示

- 证据：`CrystalPreview.tsx:146-154`

```
const measureImgRect = useCallback(() => {
  const img = imgRef.current; const container = containerRef.current;
  if (!img || !container || !img.naturalWidth || transformRef.current.scale > 1.01)
return;
  ...
}, []);
```

触发时机只有两处：`:155-161` 的 `ResizeObserver`（**观察的是 container，不是 img**）、`:162-164` 的 `useEffect(..., [url, measureImgRect])`。 - 根因：`url` 变化的 effect 会立即执行一次 `measureImgRect()`，但此刻 `<img src={新 url}>` **还没加载完成，`naturalWidth === 0`** → 直接 return。之后图片加载完成不会触发 ResizeObserver（container 尺寸没变），也没有 `onLoad` 回调 → **`imgRect` 永远停留在 null**。 - 而 `imgRect` 是 `cropFrameRect`（裁剪框矩形，`:181-193`）和三分网格定位的唯一依赖。`imgRect === null` → `cropFrameRect` 返回 null → **构图模式下裁剪框和三分网格都不渲染**，用户拖动手柄时看不到框在哪。 - **强对照证据**：`WatermarkExportPage.tsx:118-142` 是同一套 contain 测量逻辑，那里**明确绑定了** `el.addEventListener('load', measure)` 并在 cleanup 里 removeEventListener。说明作者知道必须绑 load，`CrystalPreview` 这一处漏了。 - 唯一能意外救回的路径是转屏/键盘弹出触发 container resize。所以症状是「有时候有网格有时候没有」——典型的难复现 bug。 - 修复：给 `<img>` 加 `onLoad={measureImgRect}`。

### P-2 · P1 · `onPointerCancel` 复用 `onPointerUp`，被系统打断的手势会误触发双击缩放

- 证据：`CrystalPreview.tsx:608` → `onPointerCancel={onPointerUp}`
- 根因：`onPointerUp:543-578` 内含完整的双击检测：

```
const wasSingleTap = (g.mode === 'pan' || g.mode === 'cropDraw') && !g.moved;
...
if (wasSingleTap) {
  const prev = lastTap.current;
  if (prev && now - prev.t < DOUBLE_TAP_MS && dist(prev, ...) < TAP_SLOP * 2) { ...
zoomAt(...) }
  else { lastTap.current = { t: now, x: e.clientX, y: e.clientY }; }
}
```

pointer cancel 的语义是「浏览器/系统接管了这个指针，手势作废」（滚动、系统手势、来电、通知栏下拉）。此时 `g.moved` 常常仍是 `false` → **被当成一次有效 tap 记进 `lastTap`**。 - 影响：用户在预览区上下滑动翻看（触发 cancel），紧接着一次普通点击就会被判定为「双击」→ **画面意外跳到 2x 缩放**。表现为「预览 randomly 自己放大了」。 - 修复：`onPointerCancel` 用独立的 handler，只做 `pointers.delete` + 状态复位，**不参与 tap 判定**。

### P-3 · P2 · 长按看原图：`start` 不先清除旧 timer，多点触控下会卡死在原图

- 证据：`crystalShared.tsx` → `useShowOriginalHold`

```
const start = useCallback(() => {
  holdTimer.current = window.setTimeout(() => { holdActive.current = true; ...
setEditor({ showOriginal: true, previewOverride: ... }); }, showDelayMs);
}, [setEditor, showDelayMs]);

const end = useCallback(() => {
  if (holdTimer.current !== null) { clearTimeout(holdTimer.current);
holdTimer.current = null; }
  if (holdActive.current) { holdActive.current = false; setEditor({ showOriginal:
false, previewOverride: null }); }
}, [setEditor]);
```

- 根因：`start` 直接覆写 `holdTimer.current`，**不先 clearTimeout**。两根手指先后按下（或手指按下未抬起又触发一次 pointerdown）时：第一个 timer 引用丢失但仍在计时，`end` 只能清掉第二个。第一个 timer 到点后执行 `holdActive.current = true; setEditor({showOriginal: true, ...})`，而此时 `end` 已经跑完并把 `holdActive` 置回 false → **后续任何松手都不会复位 `showOriginal`**。
- 影响：预览**永久卡在原图**，用户改任何参数都看不到效果，只能退出编辑域重进。代码注释自己就提到了「多点触控切换流程导致卸载」这个场景，并加了卸载兜底 —— 但兜底只在**组件卸载**时生效，同一界面内的卡死救不回来。
- 修复：`start` 开头先 `if (holdTimer.current !== null) clearTimeout(holdTimer.current);` 并用 pointerId 记录是哪根手指触发的，`end` 只响应对应 pointerId。

### P-4 · P2 · `cropResize` 手势期间新触点会污染 pointer 表，状态不收敛

- 证据：`CrystalPreview.tsx:460-479`

```
const onPointerDown = useCallback((e) => {
  if (compareRef.current || !urlRef.current) return;
  if ((e.target as HTMLElement).closest('button')) return;
  pointers.current.set(e.pointerId, {...});        // ← 已经写入
  e.currentTarget.setPointerCapture(e.pointerId);  // ← 已经捕获
  const g = gesture.current;
  if (g.mode === 'cropResize') {
    // D-CP-02: 手柄调整进行中忽略新触点（单手势互斥），仅记录点位供 up 清理
    return;
  }
  ...
```

- 根因：注释说「仅记录点位供 up 清理」，但 `onPointerUp:549` 的收敛分支是 `if (pointers.current.size === 1 && g.mode === 'pinch')` —— **mode 是 `cropResize` 不是 `pinch`，这个分支进不去**。抬指顺序若让 size 从 2→1，会跳过 `size === 0` 的复位分支，`gesture.mode` 停留在 `cropResize`、`setGesturing(true)` 不复位。

- 影响：裁剪手柄调整过程中误触第二根手指（移动端非常常见），之后手势系统状态错乱——`gesturing` 常真会持续压制过渡动画，且下一次单指拖动可能被错误地当成手柄调整。
- 修复：`cropResize` 分支直接 `return` **之前**不要写入 `pointers` / 不要 `setPointerCapture`；或在 `onPointerUp` 里补 `mode === 'cropResize'` 的收敛分支。

### P-5 · P3 · 双指缩放后 `clampPan` 用 `naturalWidth` 而非 contain 适配尺寸，边界钳制在旋转图上失准

- 证据：`CrystalPreview.tsx:196-204`

```
const fit = Math.min(rect.width / img.naturalWidth, rect.height / img.naturalHeight);
const maxX = Math.max(0, (img.naturalWidth * fit * next.scale - rect.width) / 2);
```

- 根因：`img.naturalWidth/naturalHeight` 是**未定向**的原始像素尺寸。而裁剪/网格那一套全程用 `orientedSize(w, h, orientationSteps)`（`getOrientedDimensions`）处理旋转后的宽高。横构图照片旋转 90° 后，`naturalWidth > naturalHeight`，但实际显示是竖的 → `fit` 与 `maxX/maxY` 算反。
- 影响：旋转过的图片放大后**平移边界钳制错误**——可以拖出画面留黑边，或者反过来拖不到边缘。仅在「先旋转 90°/270° 再双指放大平移」时出现，属于条件触发的边缘 bug。
- 修复：`clampPan` 改用 `orientedSize(...)` 的结果。

---

## 四、导出模块（5 项）

### E-1 · P0 · Android 从 SAF 导入的图片，导出文件名是 URI 尾段乱码

- 证据：`WatermarkExportPage.tsx:236-238`

```
const originalFilename = selectedImage.path.split(/[\\/]/).pop() || '';
const stem = originalFilename.substring(0, originalFilename.lastIndexOf('.')) ||
originalFilename;
const outputFileName = `${stem}${suffix}.${format === 'jpeg' ? 'jpg' : 'png'}`;
```

- 交叉对照（**这是本条最有力的证据**）：仓库里另外 5 处处理文件名时**全部**做了 content URI 解析，只有导出页这一处没做——

  ▸ `hooks/useEditorActions.ts:254-257`

  ```
  const name = isAndroid && path.startsWith('content://')
    ? await invoke<string>('resolve_android_content_uri_name', { uriStr: path })
    : path.split(/[\\/]/).pop() || 'LUT';
  ```

  ▸ `components/mobile/CrystalPresetsPanel.tsx:987`

  ▸ `components/mobile/CrystalCategoryPanel.tsx:2122`

  ▸ `components/ui/LUTControl.tsx:172`

  ▸ `hooks/useFileOperations.ts:418`

- 后端确认落地：`export_processing.rs:1648-1656` → `output_path.file_name()` 直接作为 `save_image_bytes_to_android_gallery(file_name, ...)` 的 `_display_name` 写入 MediaStore。**前端算错的这个名字就是用户在相册里看到的文件名。**
- 影响：Android 上通过系统相册/文件管理器（SAF）导入的图片，`selectedImage.path` 是 `content://com.android.providers.media.documents/document/image%3A1234` 这类 URI。`split(/[\\/]/)` 后得到 `image%3A1234`，`lastIndexOf('.')` 返回 −1 → `stem` = 整个尾段 → 导出文件名变成 `image%3A1234_crystal.jpg`。**用户在相册里看到一堆 URL 编码乱码文件名，**

**无法辨认是哪张照片**。这是导出模块的核心输出物，且是 Android 端**最主要的图片导入路径**，影响面接近全量用户。

- 修复：与其余 5 处对齐——`isAndroid && path.startsWith('content://')` 时先 `await invoke('resolve_android_content_uri_name', { uriStr: path })` 拿 DISPLAY_NAME，再做 stem 截取。

### E-2 · P1 · 导出成功后状态滞留，重进导出页一直显示「导出完成」

- 证据：`WatermarkExportPage.tsx:150-159`

```
const prevExportStatusRef = useRef<Status | null>(null);
useEffect(() => {
  const prev = prevExportStatusRef.current;
  prevExportStatusRef.current = exportState.status;
  if (exportState.status === Status.Success && prev !== null && prev !==
Status.Success) {
    haptic([10, 40, 10]); toast.success(t('crystal.export.done'));
  }
}, [exportState.status, t]);
```

以及页面底部 `{exportState.status === Status.Success && (<div ...>{t('crystal.export.done')}</div>)}`。 - 根因：`prevExportStatusRef` 的守卫**只挡住了 toast**（注释 F9 明确说就是为了防重挂载误弹）。但底部那行常驻的成功文案**没有任何守卫**，直接读 store 里的 `exportState.status`。 - 影响：导出成功 → 返回编辑域 → 再次进入导出页，`exportState.status` 仍是 `Success`（store 是全局的，不会因组件卸载而重置）→ **页面底部一进来就挂着绿色的「导出完成」**，而这次用户什么都没做。会被误读为「刚才又导出了一次」或「导出状态错乱」。 - 修复：进入导出页时（或 `handleExport` 开始时）把 `exportState` 重置为 Idle；或底部文案也加与 toast 相同的迁移守卫。

### E-3 · P1 · 长按对照原图时同一张大图被渲染两层，移动端内存翻倍

- 证据：`WatermarkExportPage.tsx` 预览区

```
<img ref={previewImgRef}
     src={holdOriginal ? (originalPreview ?? finalPreviewUrl) :
finalPreviewUrl} ... />
...
{holdOriginal && originalPreview && (
  <img className="cx-export-origin" src={originalPreview} ... />
)}
```

- 根因：`holdOriginal` 为真时，**主图的 src 已经切成了 `originalPreview`，叠加层又用同一个 `originalPreview` 再渲染一遍**。两层完全重叠、内容相同。 代码注释揭示了成因：「审计修复：此前仅翻本地 holdOriginal……现与底栏原图键同机制：按住写 previewOverride」。也就是新机制（改 `previewOverride` 让主管线把原图渲进 `finalPreviewUrl`）加上去了，**但旧的 `originalPreview` 叠加层没有一并删除**。两套实现叠在一起。
- 影响：导出页长按对照原图时，一张全尺寸原图在 DOM 里存在两份 → 移动端解码内存翻倍。导出页本来就是内存压力最大的场景（`finalPreviewUrl` + 导出缓冲），这里再翻倍很容易触发 WebView 回收或 OOM，表现为**长按对照时白屏/闪退**。
- 附带问题：主图 `src` 在 `finalPreviewUrl` 与 `originalPreview` 之间切换，会触发 `useEffect([finalPreviewUrl])` 之外的重新解码；叠加层的出现/消失还伴随一次布局抖动。
- 修复：删除 `cx-export-origin` 叠加层，只保留 `previewOverride` 这一套机制（与底栏原图键一致）。

### E-4 · P2 · 签名输入每敲一个字符同步写一次 localStorage

· 证据：`WatermarkExportPage.tsx`

```
<input className="cx-input" value={signatureDraft}
       onChange={(e) => commitSignature(e.target.value)} maxLength={40} />
```

```
const commitSignature = (value: string) => { setSignatureDraft(value);
updatePrefs({ signature: value }); };
```

`useWmPrefs.update` 内部 `localStorage.setItem(PREFS_KEY, JSON.stringify(next))` —— **同步阻塞 IO**。 - 影响：中文输入法在组合期（拼音未上屏）会高频触发 onChange，每次都做一次 `JSON.stringify` + 同步 `setItem`。低端 Android 机上**输入签名会明显卡顿/掉帧**。 - 修复：`setSignatureDraft` 立即更新（受控输入不卡），`updatePrefs` 用 debounce（300–500ms）或改到 `onBlur` 提交。

### E-5 · P3 · 导出格式的持久化策略自相矛盾

· 证据：`WatermarkExportPage.tsx:73-75`

```
const [format, setFormat] = useState<'jpeg' | 'png'>('jpeg');
const [quality, setQuality] = useState(100);
const [showInfoBarFrame, setShowInfoBarFrame] = useState(true);
```

而 `fontScale` / `fontFamily` / `filenameSuffix` / `favoriteTemplateId` / `defaultTemplateId` 走 `prefs`（localStorage 持久化）。 - 影响：用户每次进导出页，格式都回到 JPEG、画质都回到 100、信息栏都回到开——**但这几项恰恰是最稳定的个人偏好**（习惯导 PNG 的用户每次都要重新点）。而相对次要的字号、后缀反而被记住了。 - 附带：`defaultTemplateId` 被持久化并用于 `useState(prefs.defaultTemplateId ?? 'pro')` 初值，但 `selectedTemplateId` 本身不持久化——用户「设为默认」后当次会话内选中的模板与默认模板会脱钩，语义容易混淆。 - 修复：把 `format` / `quality` / `showInfoBarFrame` 一并纳入 `CrystalWatermarkPrefs`。

---

## 五、人像模块（3 项）

### PT-1 · P1 · `findContainer` 匹配任意 subMask，但 `maskReady` / `removeContainer` 只看 `subMasks[0]`

· 证据：`crystalShared.tsx` → `useCrystalAiMasks`

```
const findContainer = useCallback((type: Mask): AiContainer | null => {
  const { adjustments } = useEditorStore.getState();
  return (adjustments.masks || []).find((c) => c.subMasks.some((sm) => sm.type ===
type)) || null;
}, []);

const maskReady = useCallback((type: Mask) => {
  const container = findContainer(type);
  return Boolean(container?.subMasks[0]?.parameters?.maskDataBase64);   // ← [0]
}, [findContainer]);

const removeContainer = useCallback((type: Mask) => {
  const container = findContainer(type);                                // ← some()
  if (!container) return;
  setAdjustments((prev) => ({ ...prev, masks: (prev.masks||[]).filter((c) => c.id !==
container.id) }));
}, [findContainer, setAdjustments]);
```

- 根因：查找用 `.some(sm => sm.type === type)`（**任意位置**匹配），读取和删除用 `subMasks[0]`（**只看第一个**）。两者口径不一致。
- 影响：容器内有多个 subMask、且目标类型不在第 0 位时——
  - `maskReady` 检查错对象 → **蒙版已经生成好了，UI 仍显示「未识别」，按钮一直停在可点击的生成态**；反之也可能显示已就绪但实际没有数据
  - 人像模块同时存在 `findContainerByKind`（按 `parameters.retouchKind` 区分磨皮/牙齿/眼部），说明**多个 AiFace 容器并存是被支持的设计**。`removeContainer(type)` 按 type 匹配会命中第一个含 AiFace 的容器，**可能删掉用户另一个精修算子的容器**
- 修复：`findContainer` 返回值应同时给出命中的 subMask 索引；`maskReady` / `removeContainer` 按该索引操作。按 type 删除的入口应统一改走 `removeContainerByKind`。

### PT-2 · P1 · 美型值调回 0 后丢失宿主容器，反复调整会无限堆积 AiFace 容器

- 证据：`crystalShared.tsx` → `findLiquifyHost`

```
const holder = aiFaces.find((c) => {
  const liq = c.subMasks[0]?.parameters?.liquify as { slimFace?: number; jawline?:
number } | undefined;
  return Boolean(liq && ((liq.slimFace ?? 0) > 0 || (liq.jawline ?? 0) > 0));   // ←
只认 > 0
});
if (holder) return holder;
return aiFaces.find((c) => !c.subMasks[0]?.parameters?.retouchKind) ?? null;
```

以及 `CrystalCategoryPanel.tsx:1478` → `const liquifyHost = findLiquifyHost();`，调用方在拿不到 host 时走 `createLiquifyHost()` **新建一个容器**。 - 根因：宿主的识别条件是「`slimFace > 0` 或 `jawline > 0`」。用户把瘦脸/下颌线**调回 0**（这是很自然的操作：试一下效果不满意，拖回原点），判定条件立刻不成立。 - 影响链： 1. 第一层兜底 `aiFaces.find(c => !c.subMasks[0]?.parameters?.retouchKind)` 会命中「无 retouchKind 的 AiFace 容器」——若之前的美型宿主还在，能兜住 2. 但若用户此前是先开了磨皮（带 retouchKind）、再调美型，兜底找不到无 retouchKind 的容器 → 返回 null → **`createLiquifyHost()` 又新建一个 AiFace 容器** 3. 反复「调上去 → 调回 0 → 再调上去」会不断堆积容器。`masks` 数组无限增长，渲染管线每个容器都要跑一遍人脸液化 → **越用越卡，且效果互相叠加**（多个液化补丁作用在同一张脸上） - 与 G-2 叠加：每个容器还各带一份 `maskDataBase64`，内存放大倍数同步上升。 - 修复：宿主的识别应基于**显式标记**（例如 `subMask.parameters.isLiquifyHost === true`），而不是靠「当前值是否 > 0」这种会随用户操作消失的推断。

### PT-3 · P2 · 蒙版尺寸在图片信息缺失时兜底 1000×1000，导致蒙版错位

- 证据：`crystalShared.tsx` → `ensureContainer` 与 `createLiquifyHost` 各有一处

```
const subMask = createSubMask(type, {
  width: selectedImage?.width ?? 1000,
  height: selectedImage?.height ?? 1000,
});
```

- 根因：`selectedImage` 尚未就绪（或加载失败）时用 1000×1000 兜底，**但蒙版数据的坐标系必须与实际图片像素尺寸一致**。
- 影响：一张 6000×4000 的照片，若在 `selectedImage` 就绪前触发了人脸/天空/前景蒙版生成，蒙版按 1000×1000（还是正方形）的坐标系生成 → **蒙版与实际画面完全错位**，磨皮磨到背景、天空替换选中地面。而且这个错误尺寸会被写进 adjustments 并进撤销栈与 sidecar，**切回这张图后依然是错的**。

- 前置条件在移动端并不罕见：`awaitImageReady` 有 10s 超时，超时返回 false 后各调用点会早退（这部分写对了），但**没有走 `awaitImageReady` 的蒙版入口**（如直接点美型滑杆触发 `createLiquifyHost`）没有这层保护。
- 修复：`selectedImage?.width` 缺失时应**直接拒绝创建容器并提示「图片加载中」**，而不是用假尺寸兜底。1000×1000 这个魔法数字应当删除。

---

## 六、编辑壳 / 底栏导航（2 项）

### S-1 · P1 · 重置的 sidecar 写入失败被完全静默，重进图片旧编辑复活

- 证据：`CrystalMobileApp.tsx:404-413`

```
const handleReset = useCallback(() => {
  const sel = useEditorStore.getState().selectedImage;
  debouncedSetHistory.cancel();
  const resetData = { ...INITIAL_ADJUSTMENTS, aiPatches: [] } as Adjustments;
  resetHistory(resetData);
  setEditor({ adjustments: resetData, showOriginal: false, previewOverride: null });
  if (sel) {
    invoke(Invokes.ResetAdjustmentsForPaths, { paths: [sel.path] }).catch(() => {});
  }
}, [resetHistory, setEditor]);
```

- 根因：内存态（history + editor store）**同步立即重置**，磁盘态（sidecar）**异步且 `.catch(() => {})` 吞掉一切错误**。两者没有任何一致性保证，UI 也不等待磁盘结果。
- 影响：sidecar 写入失败时（Android 上 `content://` 路径的 sidecar 写入涉及 SAF 持久化授权，失败并不罕见）——
  - 用户看到画面立刻恢复原图、撤销栈已重建 → **认为重置成功了**
  - 但磁盘上旧的编辑参数还在
  - 退出编辑域、重新点开这张图 → **所有被「重置掉」的编辑全部复活**
- 这是「UI 说成功、实际没成功」的典型静默数据不一致，且用户无法察觉。
- 附带：`debouncedSetHistory.cancel()` 只取消了历史推送，**没有取消 `debouncedSave`**（`useEditorActions.ts:34-39`，300ms）。若重置发生在一次滑杆调整后的 300ms 窗口内，`debouncedSave` 仍会用**旧参数**触发一次 `SaveMetadataAndUpdateThumbnail` —— 与 `ResetAdjustmentsForPaths` 形成写竞争，谁后到谁生效。
- 修复：`await` sidecar 重置结果，失败时 toast 报错并回滚内存态（或至少标记为「未保存」）；同时在重置前 `debouncedSave.cancel()`。

### S-2 · P2 · 返回图库时不清理 `compareView`，清理只挂在卸载路径上

- 证据：`CrystalMobileApp.tsx:308-315`

```
useEffect(() => () => {
  useEditorStore.getState().setEditor({ compareView: 'off', showOriginal: false,
previewOverride: null });
  useImmersiveCropStore.getState().exit();
}, []);
```

而顶栏返回按钮 `CrystalTopBar onBack={onBackToLibrary}` 与底栏返回键 handler 末行 `onBackToLibrary()` 都是**直接调用 prop 回调**，没有先做这个清理。 - 根因：注释写的是「离开编辑域（切导出/回图库/卸载）时强制退出对比态」，但实现只覆盖了**卸载**这一条路径。「切导出」由 `goExport` 里的 `setCompareOpen(false)` 覆盖，而**「回图库」完全没覆盖**——它依赖父组件卸载 `CrystalMobileApp`。 - 影响：若父组件在返回图库后不卸载本组件（例如用 CSS 隐藏、

或保留在路由栈里），则 compareView 残留 'horizontal' → 主管线持续渲染 originalPreview（**多一张全尺寸图的 GPU 渲染与内存占用**），且 store 里 showOriginal/previewOverride 可能残留。下次进入编辑域时对比态直接是开的。 - 待验证：我未读父组件（onBackToLibrary 的提供方），**如果它确实会卸载 CrystalMobileApp，则本条不成立**。但即便如此，把清理只放在卸载 effect 里也是脆弱的隐式契约。 - 修复：把清理逻辑抽成 leaveEditDomain()，在 onBackToLibrary 的两个调用点显式调用，不依赖卸载时机。

---

## 七、滑杆基础组件（2 项）

### SL-1 · P2 · 双击复位在移动端不可靠，且可能与页面缩放冲突

- 证据：crystalShared.tsx → CrystalRangeSlider

```
<div className="relative h-9 flex items-center" onDoubleClick={() =>
onChange(defaultValue ?? 0)}>
  ...
  <input type="range" ... style={{ height: 36, touchAction: 'pan-y' }} />
```

- 根因：
  1. onDoubleClick（dblclick）在移动端 WebView 里依赖浏览器的双击手势识别，而这个手势**默认被解释为「双击缩放页面」**。容器 div 没有设置 touch-action: manipulation，内层 input 是 pan-y（明确允许纵向平移，也就没有禁用双击缩放）
  2. 注释显示这里刚修过一轮（「自检修复（预设-强度滑杆）：双击复位此前硬编码归 0」），修的是**复位目标值**，没有修**双击本身在触屏上不可靠**这个更底层的问题
- 影响：「双击滑杆复位默认值」这个交互在 Android 上可能完全不生效，或者生效的同时把整个页面缩放了。用户唯一的复位手段失效。
- 修复：外层容器加 touch-action: manipulation（禁双击缩放但保留点击/平移）；更稳妥的做法是不依赖 dblclick，改为**长按数值标签**或**点击数值弹出输入框并附带「恢复默认」按钮**（aria-label 已经是 ui.slider.clickToEdit，说明点击数值编辑的能力本就存在）。

### SL-2 · P3 · 全负区间滑杆的填充条起点算错

- 证据：crystalShared.tsx

```
const frac = max !== min ? (value - min) / (max - min) : 0;
const zeroFrac = max !== min && min < 0 && max > 0 ? -min / (max - min) : 0;
const left = Math.min(frac, zeroFrac) * 100;
const width = Math.abs(frac - zeroFrac) * 100;
```

- 根因：zeroFrac 的条件要求 min < 0 && max > 0（零点在区间内）。当区间**全为负**（min < 0 && max <= 0）时 zeroFrac = 0，而 frac = 0 对应的是滑杆最左端（即 min）。此时「从 0 开始画填充条」的语义是错的——0 根本不在这个滑杆的取值范围内。
- 影响：left = min(frac, 0) * 100 = 0、width = |frac - 0| * 100 = frac * 100 → 填充条从最左端画到当前值，看起来像「从 min 累积」而非「相对零点的偏移」。对于色彩偏移、色温微调这类可能出现全负区间的滑杆，**视觉反馈与实际语义不符**（不过金色填充条的方向仍然单调正确，所以是视觉语义瑕疵而非功能错误）。
- 修复：zeroFrac 分三种情况处理——零点在区间内用 -min/(max-min)；max <= 0 时零点在右端之外，应取 1；min >= 0 时取 0。

---

## 八、EXIF 信息栏（1 项）

### EX-1 · P2 · EXIF 摘要未做单位与格式归一，原始串会被烧录进导出图

- 证据：`crystalShared.tsx` → `summarizeExif`

```
focal:    EXIF_STR(e.FocalLength) || EXIF_STR(e.FocalLengthIn35mmFilm) || '',
aperture: EXIF_STR(e.FNumber) || EXIF_STR(e.ApertureValue) || '',
shutter:  EXIF_STR(e.ExposureTime) || '',
iso:      EXIF_STR(e.PhotographicSensitivity) || ... || '',
```

`EXIF_STR` 只做 `typeof v === 'string' ? v : typeof v === 'number' ? String(v) : ''`，**没有任何单位、分数、精度归一**。拼接处 `WatermarkExportPage.tsx:96-98`：

```
const exifParams = [exif.focal, exif.aperture, exif.shutter, exif.iso ? `ISO
${exif.iso}` : '']
  .filter(Boolean).join(' ');
```

- 根因：函数注释声称输出「与设计稿『E-M10MarkIV ｜ CRYSTAL ｜ 38mm f/5.6 1/200s ISO 200』同构」，但代码里**只有 ISO 加了前缀**，其余三项都是裸值：
  - `FocalLength` 后端（kamadak-exif）可能给出 `38.0`、`38`、`380/10` 或 `38.0 mm` → 显示成 `380/10` 这种分数串，或缺少 `mm`
  - `FNumber` 给出 `5.6` 或 `560/100` → **缺少 `f/` 前缀**
  - `ExposureTime` 给出 `0.005`、`1/200` 或 `0.005 s` → 长曝光时显示 `30` 而不是 `30s`，短曝光时可能显示 `0.005` 而不是 `1/200`
  - `ApertureValue` 作为 `FNumber` 的兜底时，它是 APEX 单位（如 `5` 表示 f/5.6），**与 FNumber 数值语义完全不同却直接混用**
- 影响分两层：
  1. 导出页预览的白条显示格式不规范（视觉问题）
  2. **更严重**：`handleExport` 传了 `exif_frame: { enabled: true, brand, showExif: true }`，而 Rust 端 `export_processing.rs:1378-1379` 会 `apply_exif_frame(&mut image, frame, path)` —— **信息栏是烧录进导出图片像素的**。也就是说这些未归一的原始串会**永久写进用户导出的成品照片**，无法事后修改。
- 待确认：我未读 `apply_exif_frame`（`export_processing.rs:625`）的实现，**若后端自己重新从 EXIF 读取并格式化，则前端这段 `exifParams` 只影响预览、不影响成品**，本条降级为「预览与成品不一致」（WYSIWYG 破裂，仍是 bug，但严重度下降）。这一点需要看后端实现或真机导出对比才能定级。
- 修复：`summarizeExif` 内做归一——焦距补 `mm` 并把 `a/b` 分数求值；光圈补 `f/` 前缀；快门按 `<1s → 1/N、>=1s → Ns` 格式化；`ApertureValue` 兜底时做 APEX → f-number 换算（`f = 2^(Av/2)`）。**并确认前后端使用同一套格式化逻辑**，否则预览与成品必然不一致。

---

## 九、存疑待验证（3 项，不计入 24）

| # | 疑点 | 需要的验证手段 |
|---|---|---|
| V-1 | `CrystalPreview.tsx:456` 的 `onHandlePointerDown` deps 是 `[crop, orientationSteps, selectedImage]`，**缺 `rotation`**。而 `rotateChip` / `setRotation` 都会改 rotation 并联动 crop。拉直后再拖裁剪手柄，闭包里的 crop 可能是旧值 → 手柄拖动错位 | 读 `resizeCropTo` 实现，确认它是否从 store 实时读 rotation；或真机测「拉直 5° → 拖角手柄」 |
| V-2 | G-3（`activeTools` 只增不减）是否真的影响渲染结果 | 读 `getActiveTools` / `getToolOwnersForKeys` 实现，并确认 GPU 管线（`gpu_processing.rs`）是否以 `activeTools` 为算子开关 |
| V-3 | EX-1 中未归一的 EXIF 串是否真被烧录进成品 | 读 `export_processing.rs:625 apply_exif_frame`，确认它用的是前端传来的串还是自己重读 EXIF |

---

## 修复优先级

| 顺序 | 缺陷 | 理由 |
|---|---|---|
| 1 | **G-2** base64 进撤销栈/sidecar | P0，移动端最可能的崩溃根因，且越用越严重 |
| 2 | **E-1** 导出文件名乱码 | P0，影响 Android 最主要导入路径的全量用户，产出物直接可见 |
| 3 | **G-1** VIBRATE 权限 | 一行 Manifest 修复，恢复整套触觉反馈，成本收益比最高 |
| 4 | **P-1** imgRect 无 load 触发 | 一行修复，恢复构图模块的裁剪框与三分网格 |
| 5 | **C-1** 9:16 chip 高亮错误 | 确定性可复现，两处同源代码一起改 |
| 6 | **S-1** 重置静默失败 | 数据不一致，用户无法察觉 |
| 7 | **P-2 P-3 P-4** 手势三项 | 集中在同一文件，一并重构 pointer 状态机 |
| 8 | **PT-1 PT-2 PT-3** 人像三项 | 集中在 `useCrystalAiMasks`，一并重构容器寻址 |
| 9 | **E-2 E-3** 导出状态与双重渲染 | 局部改动 |
| 10 | **G-4 G-5 C-2 C-3 E-4 E-5 SL-1 SL-2 EX-1 S-2** | 按模块批量处理 |

## 建议补充的回归测试

移动端功能层 8271 行**零测试覆盖**（仓库内无任何针对 `src/components/mobile/` 的测试文件；Android 侧 junit/espresso 依赖已在 `app/build.gradle.kts` 声明但无 androidTest 源文件）。按投入产出排序：

1. `activeAspect` 匹配逻辑 —— 纯函数，表驱动断言 6 个 chip × {正向, 逆向} 全组合（**能直接钉死 C-1**）
2. `summarizeExif` —— 纯函数，喂入 `{FocalLength: '380/10', FNumber: 5.6, ExposureTime: 0.005, ApertureValue: 5}` 等真实 EXIF 形态，断言输出格式（**钉死 EX-1**）
3. `androidBackRegistry` —— LIFO 顺序、重复注册、handler 未消费三种场景（钉死 G-4）
4. `CrystalRangeSlider` 的 `zeroFrac` —— 全负/全正/跨零三种区间的填充条几何（钉死 SL-2）
5. `useCrystalAiMasks.findLiquifyHost` —— 美型值调回 0 后仍应命中同一宿主（钉死 PT-2）
6. Testing Library：`CrystalPreview` 在 `<img>` load 事件后 `imgRect` 应非 null（钉死 P-1）
7. Espresso：真机验证触觉反馈、返回键、导出文件名

## 一句话总结

**24 项功能性缺陷，其中 3 项 P0**。最该先动的是 **G-2（base64 深度图/蒙版进 50 步撤销栈与 sidecar）** 和 **E-1（Android 导出文件名对 content URI 失效）**——前者是移动端崩溃与卡顿的主因，后者让每一个从系统相册导入的用户都拿到乱码文件名。而 **G-1（VIBRATE 权限缺失）** 只需在 Manifest 加一行，就能让全 App 30 多处触觉反馈从「完全静默失效」恢复正常。

需要提醒的是：**调色（TonePanel ~800 行）与特效（FxPanel ~930 行）两个模块本次只做了定点抽查，未逐行审计**，所以「24 项」不能理解为这两个模块已经干净。若需要把它们也覆盖到，我可以再做一轮专项。